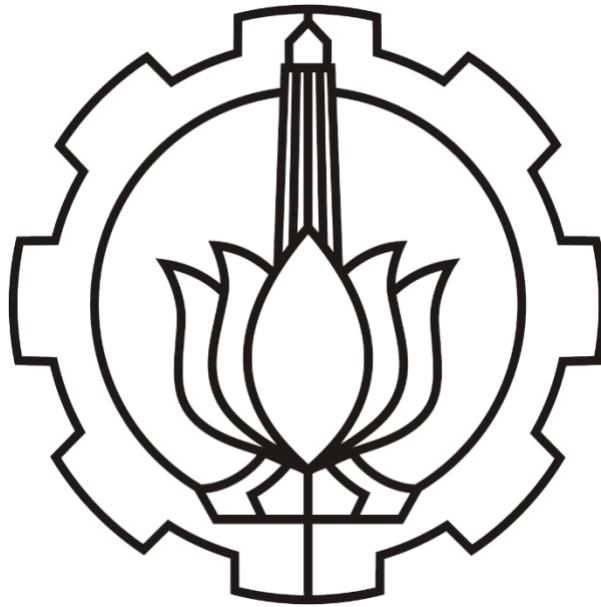


Dibuat untuk memenuhi tugas Dasar Sistem Cerdas

Fuzzy Logic



Dosen pembimbing: Fauzan Arrofiqi, S.T, M.T.

Disusun oleh:
Windy Deftia M
07311640000015

Teknik Biomedik
Institut Teknologi Sepuluh Nopember
2018

DAFTAR ISI

DASAR TEORI.....	2
Himpunan <i>Fuzzy</i>	2
Perbandingan Crisp Set dan <i>Fuzzy</i> Set	3
Semesta Pembicaraan	4
Operator Himpunan <i>Fuzzy</i>	5
DATA DAN ANALISIS	7
KESIMPULAN	13

DASAR TEORI

Logika *fuzzy* pertama kali dikembangkan oleh Lotfi A. Zadeh pada tahun 1965 melalui tulisannya mengenai Teori Himpunan *Fuzzy*. Logika *fuzzy* umumnya diterapkan pada masalah-masalah yang mengandung unsur ketidakpastian (*uncertainty*), ketidaktepatan (*imprecise*), noisy, dan sebagainya. Logika ini menjembatani bahasa mesin yang presisi dengan bahasa manusia yang menekankan pada makna atau arti (*significance*). Sebagai contoh kasus:

- Seseorang dikatakan “tinggi” jika tinggi badannya lebih dari 1,7 meter. Sedangkan orang yang mempunyai mempunyai tinggi badan 1,6999 meter atau 1,65 meter menurut persepsi manusia akan dikatakan “kurang lebih tinggi” atau “agak tinggi”.

Ketidakpastian dalam kasus –kasus yang ada disebabkan oleh ambiguitas dari pengertian “agak”, “kurang lebih”, “sedikit”, dan sebagainya

Himpunan *Fuzzy*

Himpunan klasik yang selama ini dipelajari disebut himpunan tegas atau *crisp set*. Pada himpunan ini, keanggotaan suatu unsur di dalam himpunan dinyatakan secara tegas apakah objek tersebut anggota himpunan atau bukan. Sebagai contoh:

- $A = \{0, 4, 7, 8, 11\}$, maka $7 \in A$, tetapi $5 \notin A$.

Sedangkan pada himpunan *fuzzy* dilambangkan dengan χ , yang mendefinisikan apakah suatu unsur dari semesta pembicaraan merupakan anggota suatu himpunan atau bukan:

$$\chi_A(x) = \begin{cases} 1 & , x \in A \\ 0 & , x \notin A \end{cases}$$

Contoh 1.

$X = \{1, 2, 3, 4, 5, 6\}$ dan $A \subseteq X$, yang dalam hal ini $A = \{1, 2, 5\}$.

$A = \{(1,1), (2,1), (3,0), (4,0), (5,1), (6,0)\}$

Keterangan: (2,1) berarti $\chi_A(2) = 1$; (4,0) berarti $\chi_A(4) = 0$,

Contoh 2.

V = himpunan kecepatan pelan (yaitu $v \leq 20$ km/jam)

$V = 20,01$ km/jam termasuk ke dalam himpunan kecepatan pelan?

Menurut himpunan tegas: $20,01$ km/jam $\notin V$

Menurut himpunan *fuzzy*, $20,01$ km/jam tidak ditolak ke dalam himpunan V , tetapi diturunkan derajat keanggotaannya.

Di dalam teori himpunan *fuzzy*, keanggotaan suatu elemen di dalam himpunan dinyatakan dengan derajat keanggotaan (membership values) yang nilainya terletak di dalam selang $[0, 1]$. Derajat keanggotaan ditentukan dengan fungsi keanggotaan: $\mu_A : X \rightarrow [0, 1]$

Arti derajat keanggotaan

- Jika $\mu_A(x) = 1$, maka x adalah anggota penuh dari himpunan A
- Jika $\mu_A(x) = 0$, maka x bukan anggota himpunan A
- Jika $\mu_A(x) = \mu$, dengan $0 < \mu < 1$, maka x adalah anggota himpunan A dengan derajat keanggotaan sebesar μ

Perbandingan Crisp Set dan *Fuzzy Set*

- Pada crisp set batas-batas himpunan tegas
- Pada *fuzzy set* batas-batas himpunan kabur

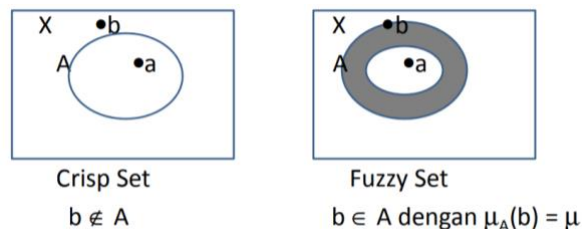


Fig 1. Perbedaan antara *fuzzy set* dengan *crisp set*

Himpunan *fuzzy* mempunyai dua atribut:

- a. Lingusitik: penamaan grup yang mewakili kondisi dengan menggunakan bahasa alami

Contoh: PANAS, DINGIN, TUA, MUDA, PELAN, dsb

- b. Numerik Numerik: nilai yang menunjukkan menunjukkan ukuran variabel variabel *fuzzy*

Contoh: 35, 78, 112, 0, -12, dsb

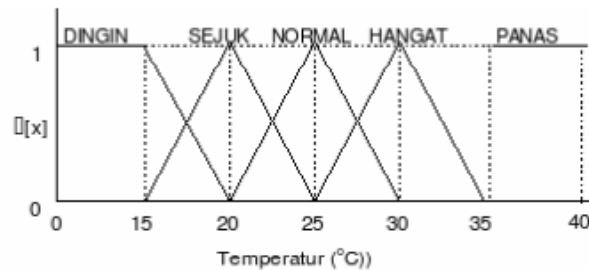


Fig 2. Contoh Himpunan fuzzy

Semesta Pembicaraan

Semesta pembicaraan adalah keseluruhan nilai yang diperbolehkan untuk dioperasikan dalam suatu variabel *fuzzy*. Semesta pembicaraan merupakan himpunan bilangan real yang senantiasa naik (bertambah) secara monoton dari kiri ke kanan. Nilai semesta pembicaraan dapat berupa bilangan positif maupun negatif.

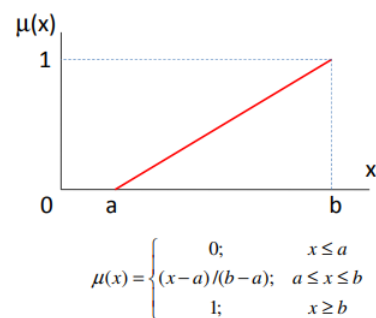
- Contoh:

Semesta pembicaraan untuk variabel umur: $[0 + \infty)$

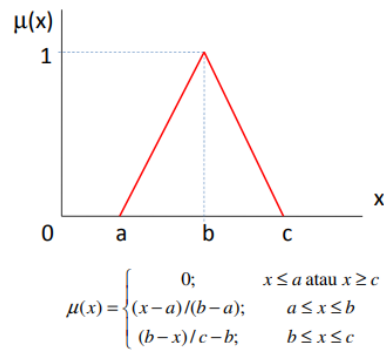
Semesta pembicaraan untuk variabel temperatur: $[] 0 40$

Fungsi Keanggotaan

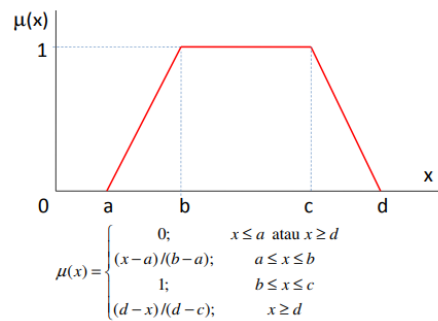
- a. Linier



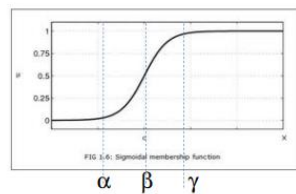
- b. Segitiga



c. Trapezium

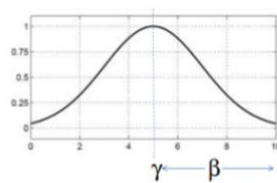


d. Sigmoid



$$S(x; \alpha, \beta, \gamma) = \begin{cases} 0; & x \leq \alpha \\ \frac{2(x-\alpha)/(\gamma-\alpha)^2}{1-2((\gamma-x)/(\gamma-\alpha))^2}; & \alpha \leq x \leq \beta \\ \frac{1-2((\gamma-x)/(\gamma-\alpha))^2}{1-2((\gamma-x)/(\gamma-\alpha))^2}; & \beta \leq x \leq \gamma \\ 1; & x \geq \gamma \end{cases}$$

e. Kurva Lonceng



$$G(x, \beta, \gamma) = \begin{cases} S(x; \gamma - \beta, \gamma - \beta/2, \gamma); & x \leq \gamma \\ 1 - S(x; \gamma, \gamma + \beta/2, \gamma + \beta); & x > \gamma \end{cases}$$

Operator Himpunan *Fuzzy*

a. AND

Operator ini berhubungan dengan operasi interseksi pada himpunan. α -predikat sebagai hasil operasi dengan operator AND diperoleh dengan

mengambil nilai keanggotaan terkecil antar elemen pada himpunan-himpunan yang bersangkutan.

Rumus α -predikat:

$$\alpha - \text{predikat} = \mu A \cap B = \min(\mu A[x], \mu B[y])$$

b. OR

Operator ini berhubungan dengan operasi union pada himpunan. α -predikat sebagai hasil operasi dengan operator OR diperoleh dengan mengambil nilai keanggotaan terbesar antar elemen pada himpunan-himpunan yang bersangkutan.

Rumus α -predikat :

$$\alpha - \text{predikat} = \mu A \cup B = \max(\mu A[x], \mu B[y])$$

c. NOT

Operator ini berhubungan dengan operasi komplemen pada himpunan. α -predikat sebagai hasil operasi dengan operator NOT diperoleh dengan mengurangkan nilai keanggotaan elemen pada himpunan yang bersangkutan dari 1.

Rumus α -predikat :

$$\mu A' = 1 - \mu A[x] \quad \mu B' = 1 - \mu B[Y]$$

DATA DAN ANALISIS

Program Untuk Operator *Fuzzy*

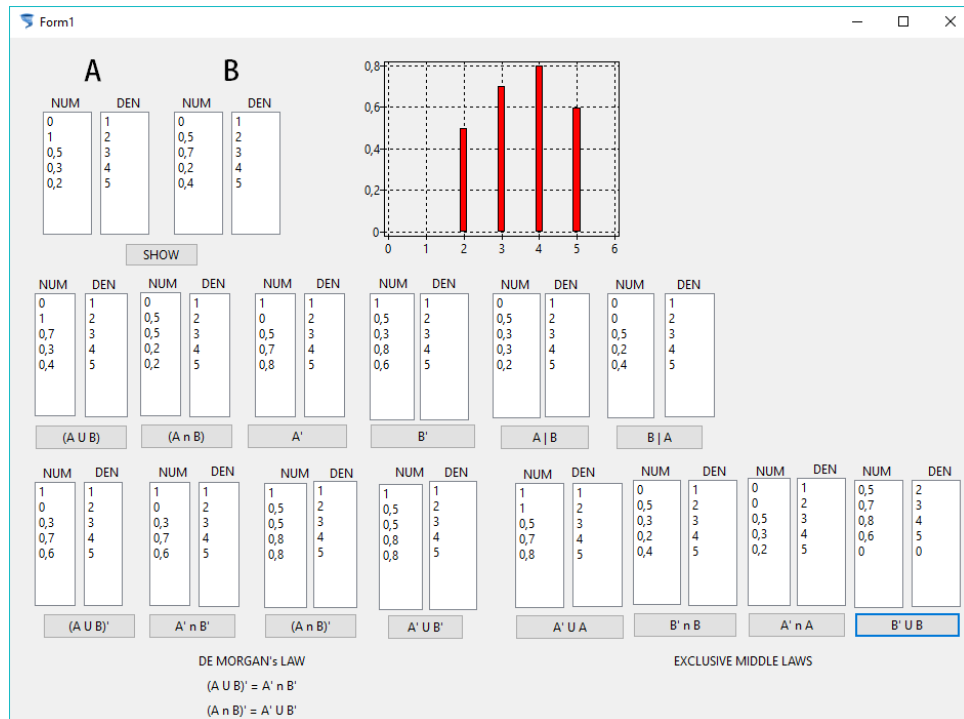


Fig 3. Interface awal program

A. Deklarasi himpunan A dan B

Pada program ini himpunan A dan B sudah dideklarasikan terlebih dahulu dengan array dua dimensi dengan baris pertama sebagai derajat keanggotaan dan baris kedua sebagai semesta pembicaraan.

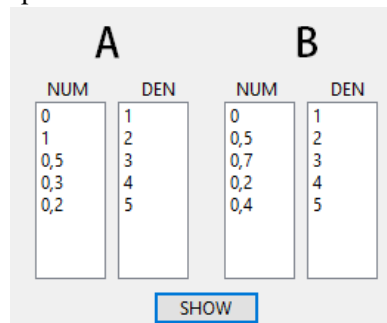


Fig 4. Deklarasi Himpunan

B. Operasi Union

Untuk operasi *union* ($A \cup B$) menggunakan logika OR yang telah dijelaskan sebelumnya, yakni dengan membandingkan $A[i,j]$ dan $B[i,j]$ kemudian diambil nilai maksimumnya. Pada program ini dibentuk sebuah *procedure* untuk memanggil fungsi *union* atau fungsi MAX dengan urutan sebagai berikut:

```
//PROCEDURE UNION//
procedure TForm1.UNION(Ain: arraybaru; Bin: arraybaru);
```

```

begin
  for i:= 0 to 1 do begin
    for j:= 0 to 4 do begin
      if (Ain[0,j] > Bin[0,j]) then begin
        Unionout[i,j] := Ain[i,j];
      end else
        Unionout[i,j] := Bin[i,j];
      end;
    end;
  end;
end;

```

Dari data yang didapatkan, hasil dari pemanggilan procedure union ini adalah:

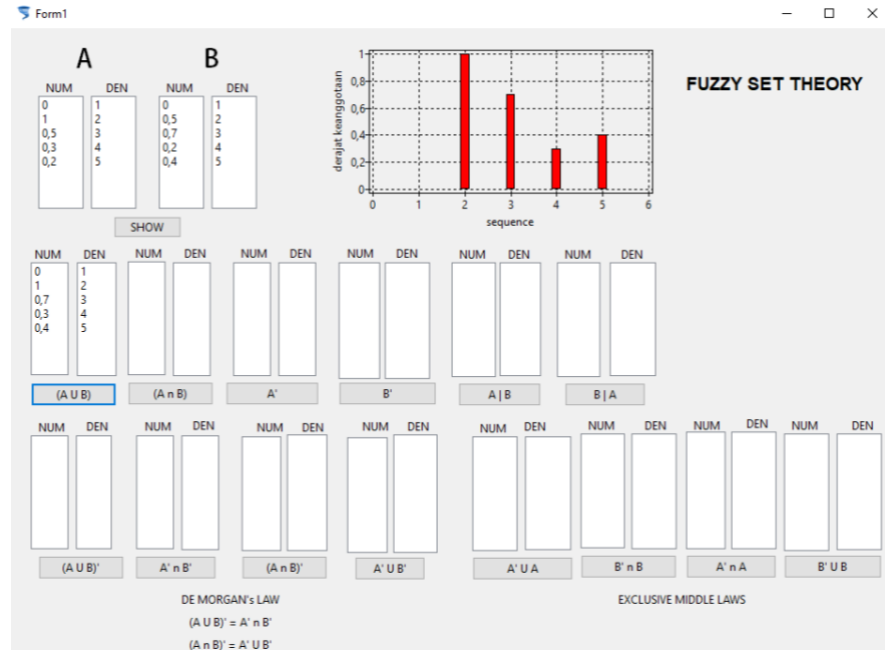


Fig 5. Hasil Union

Jika dibandingkan dengan perhitungan secara manual, hasil dari program ini sama.

C. Operasi *Intersection*

Kemudian, untuk operasi *intersection* ($A \cap B$) menggunakan logika AND yang telah dijelaskan sebelumnya yakni dengan membandingkan $A[i,j]$ dan $B[i,j]$ kemudian diambil nilai minimumnya. Seperti sebelumnya, pada program ini operasi *intersection* dibuat sebagai *procedure* agar dapat dipanggil saat mengolah operasi-operasi lainnya dengan urutan *procedure* sebagai berikut:

```

//PROCEDURE INTERSECTION//
procedure TForm1.INTERSECTION(Ain: arraybaru; Bin: arraybaru);
begin
  for i:= 0 to 1 do begin
    for j:= 0 to 4 do begin
      if (Ain[i,j] < Bin[i,j]) then begin
        Inter[i,j] := Ain[i,j];
      end else
        Inter[i,j] := Bin[i,j];
      end;
    end;
  end;
end;

```


Dari data yang didapat hasil dari program ini untuk operasi *intersection* adalah:

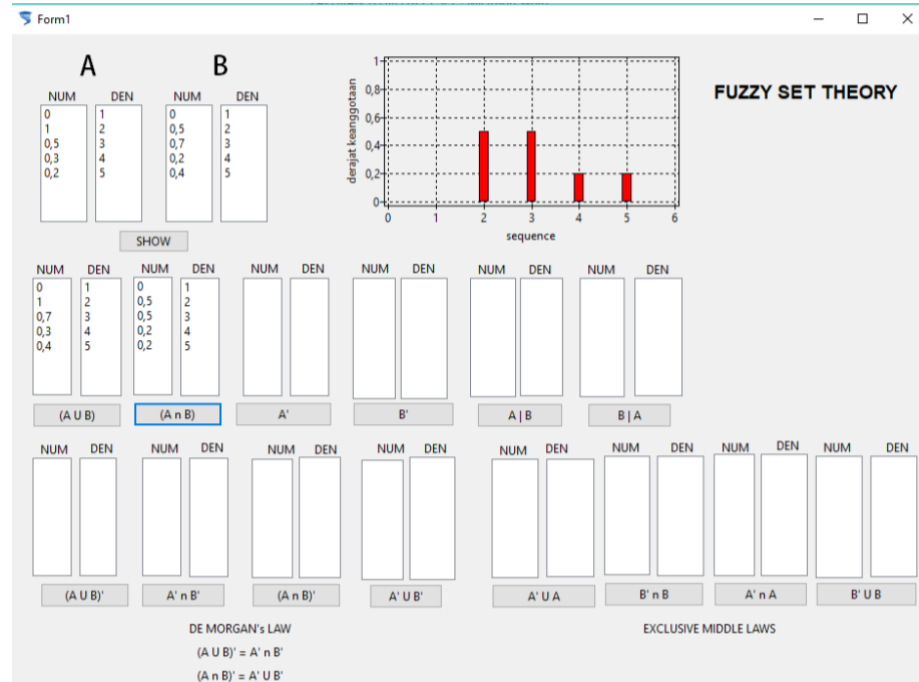


Fig 6. Hasil Intesection

Jika dibandingkan dengan hitungan manual, maka hasil dari program ini adalah sama.

D. Operasi *Complement*

Selanjutnya, untuk operasi *complement*, pada program ini menggunakan logika NOT yakni dengan mengurangi 1 dengan derajat keanggotaannya atau $(1 - \mu A[Y])$. Untuk operasi *complement* digunakan *procedure* agar operasi ini juga dapat dipanggil untuk operasi-operasi lainnya dengan urutan *procedure* sebagai berikut:

```
//PROCEDURE COMPLEMENT//
procedure TForm1.COMPLEMENT (x: arraybaru);
begin
  for i:= 0 to 1 do begin
    for j:= 0 to 4 do begin
      Com[0, j]:= 1- x[0, j];
      Com[1, j]:= x[i, j];
    end;
  end;
end;
```

Dari data yang didapat, operasi complement pada program ini adalah:

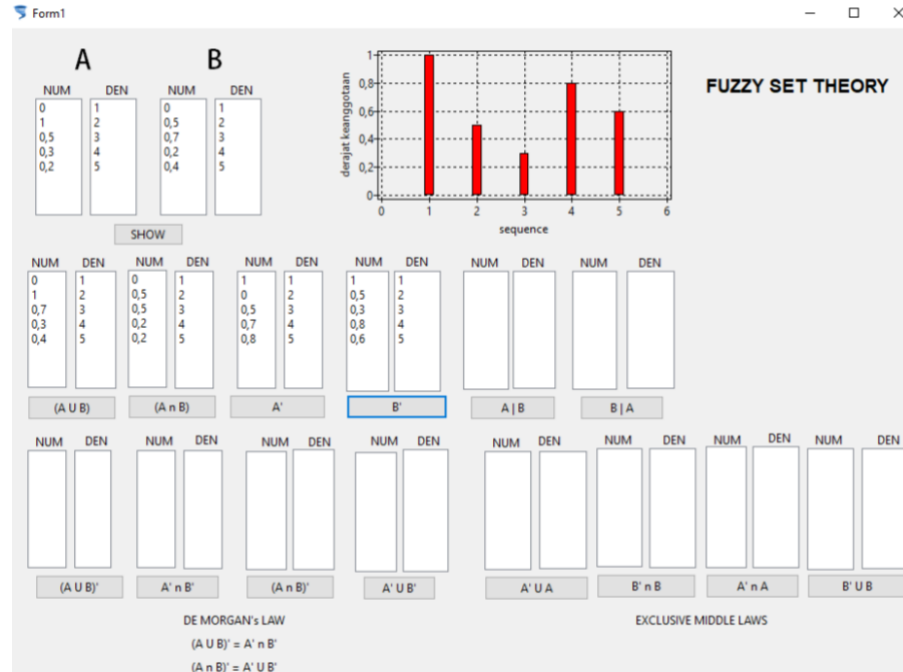


Fig 7. Hasil Complement

Hasil program ini sama dengan hasil perhitungan yang telah dilakukan sebelumnya.

E. De Morgan's Law

Setelah itu, untuk De Morgan's Law program ini dapat membuktikan bahwa $(A \cup B)' = (A' \cap B')$ serta $(A \cap B)' = (A' \cup B')$.

Karena De Morgan's Law merupakan gabungan dari operasi union, intersection dan complement maka dilakukan pemanggilan terhadap procedure-procedure yang telah dibuat sebelumnya, yang dapat disajikan dalam bentuk sebagai berikut:

```
//DE MORGAN UNION//
procedure TForm1.Button5Click(Sender: TObject);
begin
    UNION(A, B);
    COMPLEMENT(Unionout);
end;

// MORGAN INTERSECTION//
procedure TForm1.Button12Click(Sender: TObject);
begin
    COMPLEMENT(A);
    Abar:= Com;
    COMPLEMENT(B);
    Bbar:= com;
    INTERSECTION(Abar, Bbar);
end;

procedure TForm1.Button11Click(Sender: TObject);
begin
    INTERSECTION(A, B);
    COMPLEMENT(Inter);
end;
```

end;

```
procedure TForm1.Button13Click(Sender: TObject);
begin
    UNION(Abar, Bbar);
end;
```

Pada interface program didapatkan hasil sebagai berikut:

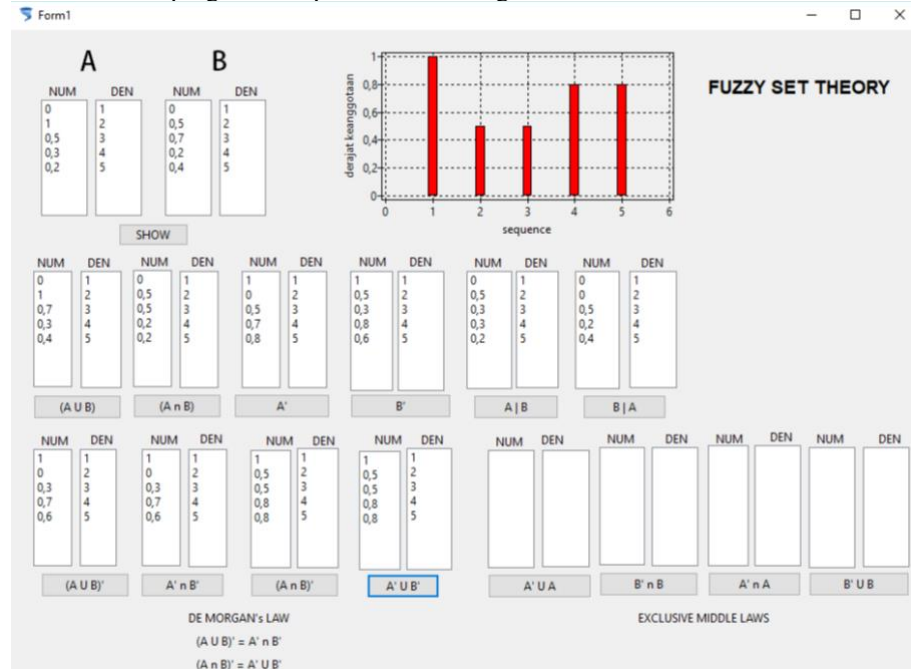


Fig 8. hasil De Morgan's Law

Dari data diatas dapat dibuktikan bahwa sesuai dengan hukum De Morgan:

$$(A \cup B)' = (A' \cap B'), \text{ dan}$$

$$(A \cap B)' = (A' \cup B').$$

F. Exclusive Middle Laws

Sedangkan yang terakhir, untuk *exclusive middle laws* program ini dapat membuktikan bahwa $(A' \cup A)$ dan $(B' \cup B)$ tidak sama dengan semesta pembicaraan serta $(A' \cap A)$ dan $(B' \cap B)$ tidak sama dengan derajat keanggotaan. Seperti hukum de Morgan, Exclusive Middle Law ini juga dilakukan pemanggilan procedure-procedure yang telah dibuat

```
//EXCLUSIVE MIDDLE LAW//
procedure TForm1.Button7Click(Sender: TObject);
begin
    COMPLEMENT(A);
    UNION(com, A);
end;

procedure TForm1.Button10Click(Sender: TObject);
begin
    COMPLEMENT(B);
    INTERSECTION(com, B);
end;

procedure TForm1.Button14Click(Sender: TObject);
begin
```

```

COMPLEMENT(A);
INTERSECTION(com, A);
end;

```

```

procedure TForm1.Button15Click(Sender: TObject);
begin
    COMPLEMENT(B);
    UNION(com, B);
end;

```

Dengan tampilan program sebagai berikut:

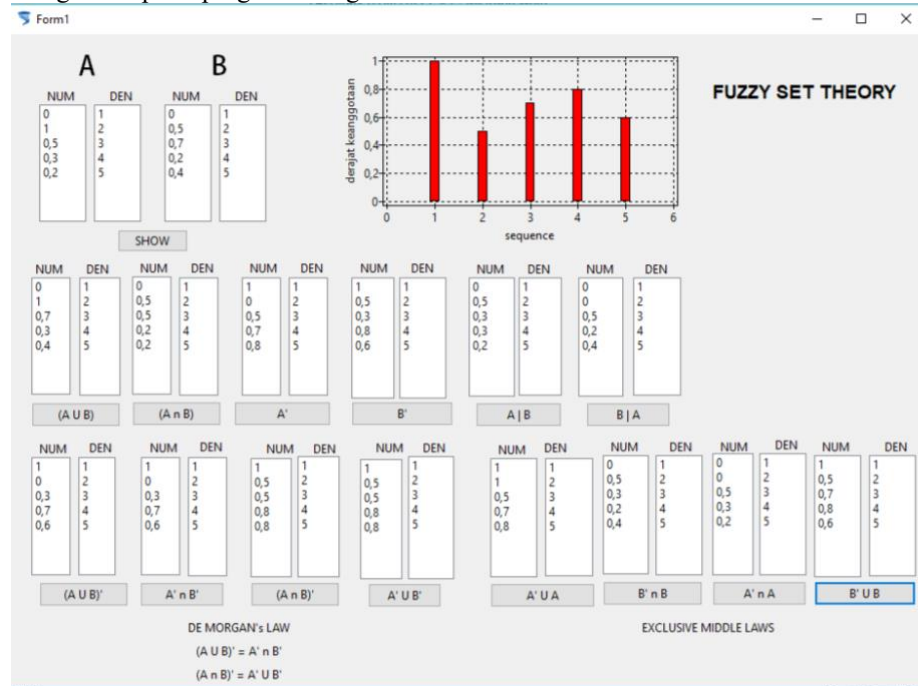


Fig 9. Hasil Exclusive Middle Laws

Dari data diatas maka program dapat membuktikan *Exclusive Middle Laws* bahwa:
 $(A' \cup A)$ dan $(B' \cup B)$ tidak sama dengan semesta pembicaraan, dan
 $(A' \cap A)$ dan $(B' \cap B)$ tidak sama dengan derajat keanggotaan.

KESIMPULAN

Dari program yang telah dibuat maka dapat ditarik kesimpulan bahwa:

1. Operasi union, intersection dan complement merupakan operasi utama. Sehingga dapat dikombinasikan untuk mendapatkan operasi-operasi lainnya
2. De Morgan's Law menjelaskan tentang:
 $(A \cup B)'$ sama seperti $(A' \cap B')$, dan
 $(A \cap B)'$ sama seperti $(A' \cup B')$.
3. Exclusive Middle Laws menjelaskan bahwa:
 $(A' \cup A)$ dan $(B' \cup B)$ tidak sama dengan semesta pembicaraan, dan
 $(A' \cap A)$ dan $(B' \cap B)$ tidak sama dengan derajat keanggotaan.